Day 110 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 6 reference

Watch: XGBoost Part 1 (of 4): Regression (StatQuest) | XGBoost in Python from Start to Finish (StatQuest)

`day110_xgboost_in_practice.py`, segment by segment, in the order `main()` uses things. Newton boosting is NumPy; the rest uses the XGBoost library, with Days 105 to 109 imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: g, h = gradient and curvature of one row · G, H = their sums · λ = penalty on leaves · ϕ_j = feature j's SHAP value

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day108_feature_importance as d108
    import day109_gradient_boosting as d109
d107, d106, d105 = d109.d107, d109.d106, d109.d105
stratified_split = d105.stratified_split
ROWS = [1000, 3000, 10000, 30000, 100000]
SCRATCH_MAX = 30000
RATES = [0.0, 0.1, 0.3, 0.5]
CONFIGS = 20
TUNE_SPLITS = 10
SPLITS = {"Titanic": 20, "Breast cancer": 20, "Digits": 6}
BASE = dict(eta=0.1, max_depth=3, seed=0, nthread=1)
```

- `import day108_feature_importance as d108` — Day 108's Titanic with its two useless columns.
- `BASE = dict(eta=0.1, max_depth=3, seed=0, nthread=1)` — XGBoost takes its settings as a dictionary; eta is its name for the learning rate.
- `SCRATCH_MAX = 30000` — the most rows given to the slow version.

Segment 1 — Newton boosting

newton_split and grow_newton

MATH

$$gain = \tfrac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda} \right], \quad leaf = -\frac{G}{H+\lambda}$$

CODE

```
def newton_split(X, g, h, lam):
    n, best = len(g), None
    G, H = g.sum(), h.sum(); parent = G ** 2 / (H + lam)
    for f in range(X.shape[1]):
        o = np.argsort(X[:, f], kind="stable"); xs = X[o, f]
        GL = np.cumsum(g[o])[:-1]; HL = np.cumsum(h[o])[:-1]
        ok = xs[1:] != xs[:-1]
        if not ok.any():
            continue
        gain = np.where(ok, 0.5 * (GL ** 2 / (HL + lam) + (G - GL) ** 2 / (H - HL + lam) -
            parent), -np.inf)
        i = int(np.argmax(gain))
        if best is None or gain[i] > best[2] + 1e-12:
            best = (f, (xs[i] + xs[i + 1]) / 2, float(gain[i]))
    return best if best and best[2] > 1e-12 else None
def grow_newton(X, g, h, idx, max_depth, lam, depth=0):
    node = {"value": float(-g[idx].sum() / (h[idx].sum() + lam))}
    if depth >= max_depth or len(idx) < 2:
        return node
    s = newton_split(X[idx], g[idx], h[idx], lam)
    if s is None:
        return node
    m = X[idx, s[0]] <= s[1]
    node.update(feature=s[0], threshold=s[1], gain=s[2], left=grow_newton(X, g, h, idx[m],
        max_depth, lam, depth + 1),
                right=grow_newton(X, g, h, idx[~m], max_depth, lam, depth + 1))
    return node
```

- `GL = np.cumsum(g[o])[:-1]` — running gradient totals, as in Day 109's search; HL does the same for curvature.
- `(G - GL) ** 2 / (H - HL + lam)` — the right side, from the totals, for every gap at once.
- `node = {"value": float(...)}` — every node knows its value, so a leaf is simply a node that stopped.

predict_lt and fit_newton

CODE

```
def predict_lt(node, X):
    out = np.zeros(len(X)); stack = [(node, np.arange(len(X)))]
    Z = X.astype(np.float32)
    while stack:
        n, idx = stack.pop()
        if "feature" not in n:
            out[idx] = n["value"]
            continue
        m = Z[idx, n["feature"]] < np.float32(n["threshold"])
        stack.append((n["left"], idx[m])); stack.append((n["right"], idx[~m]))
    return out
def fit_newton(X, y, loss, rounds, lr, max_depth, lam, Xe, base):
    y = y.astype(float); idx = np.arange(len(y)); F = np.full(len(y), base); Fe = np.full(len(Xe),
        base); trees = []
    for _ in range(rounds):
        if loss == "squared":
            g, h = F - y, np.ones(len(y))
        else:
            p = d109.sigmoid(F); g, h = p - y, p * (1 - p)
        t = grow_newton(X, g, h, idx, max_depth, lam)
        F = F + lr * predict_lt(t, X); Fe = Fe + lr * predict_lt(t, Xe); trees.append(t)
    return F, Fe, trees
```

- `X.astype(np.float32)` — XGBoost stores features in 32 bits, so the comparison must too.
- `Z[idx, n["feature"]] < np.float32(n["threshold"])` — strictly less: a row exactly on the threshold goes right, where Day 109 used <=.
- `g, h = F - y, np.ones(len(y))` — squared error: curvature 1, so a leaf is the mean residual when lambda is 0.

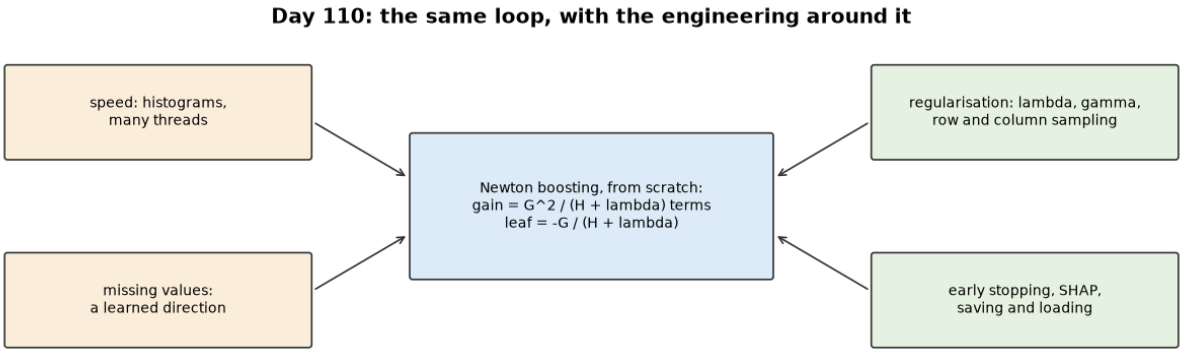
Comparing trees with XGBoost

CODE

```
def same_tree(ours, theirs):
    if "feature" not in ours:
        return "leaf" in theirs
    if "split" not in theirs or theirs["split"] != f"f{ours['feature']}":
        return False
    if abs(ours["threshold"] - theirs["split_condition"]) > 1e-4 * (1 + abs(ours["threshold"])):
        return False
    kids = {c["nodeid"]: c for c in theirs["children"]}
    return same_tree(ours["left"], kids[theirs["yes"]]) and same_tree(ours["right"],
        kids[theirs["no"]])
```

- `m.get_dump(dump_format="json", with_stats=True)` — the library's trees as text; json.loads turns each into a dictionary.
- `kids = {c["nodeid"]: c for c in theirs["children"]}` — its children are listed by id; 'yes' and 'no' say which is which.
- `a and b` — recursion on both sides; and stops at the first False.

SYSTEM DIAGRAM



The same loop, with the engineering around it.

Segment 2 — The library

fit_xgb and margin: using the library

CODE

```
def fit_xgb(X, y, params, rounds, Xv=None, yv=None, stop=None):
    d = xgb.DMatrix(X, label=y)
    ev = [(xgb.DMatrix(Xv, label=yv), "val")] if Xv is not None else []
    return xgb.train(params, d, rounds, evals=ev, early_stopping_rounds=stop, verbose_eval=False)
def margin(model, X):
    it = getattr(model, "best_iteration", None)
    return model.predict(xgb.DMatrix(X), output_margin=True, iteration_range=(0, it + 1) if it is
        not None else (0, 0))
```

- `xgb.DMatrix(X, label=y)` — the library's own data format.
- `early_stopping_rounds=stop` — stop when the last evals entry has not improved for this many rounds.
- `output_margin=True` — raw log-odds instead of probabilities.
- `iteration_range=(0, it + 1)` — use only the trees up to the best round; (0, 0) means all of them.

Timing, and a cache

CODE

```
def timed(fn):
    t = time.perf_counter(); fn(); return time.perf_counter() - t
...
Z = xgb.DMatrix(Xc[te]); cached = np.median([timed(lambda: m.predict(Z)) for _ in range(20)])
t = np.median([timed(lambda: m.predict(xgb.DMatrix(Xc[te]))) for _ in range(20)])
```

- `fn()` — fn is a function with no arguments; a lambda wraps the real call.
- `timed(lambda: m.predict(Z))` — the same Z every time: the library returns its stored answer.
- `xgb.DMatrix(Xc[te])` — a new object each time, so a real prediction is measured.

Segment 3 — The experiments

Filling gaps, inside demo_missing

CODE

```
F = Z.copy(); m = np.nanmedian(Z[tr], axis=0); m = np.where(np.isnan(m), 0.0, m)
pos = np.where(np.isnan(F)); F[pos] = m[pos[1]]
...
G = Z.copy()
for j in range(G.shape[1]):
    seen = Z[tr, j][~np.isnan(Z[tr, j])]; gaps = np.isnan(G[:, j])
    G[gaps, j] = rng.choice(seen, gaps.sum()) if len(seen) else 0.0
```

- `np.nanmedian(Z[tr], axis=0)` — each feature's median, ignoring gaps, on training rows only.
- `pos = np.where(np.isnan(F))` — two arrays: the rows and columns of every gap.
- `m[pos[1]]` — the right column's median for each gap.
- `rng.choice(seen, gaps.sum())` — gaps.sum() counts True values: one random observed value per gap.

SHAP values

MATH

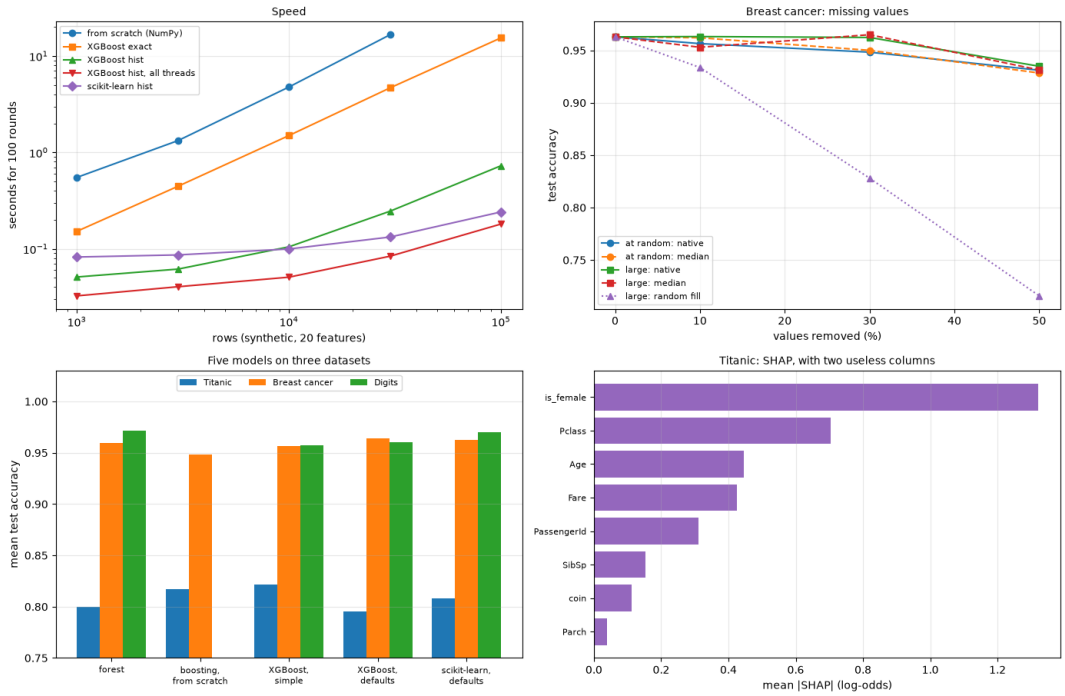
$$F(x) = base + \Sigma_j \phi_j(x)$$

CODE

```
C = m.predict(xgb.DMatrix(X[te]), pred_contribs=True)
gap = np.abs(C.sum(1) - margin(m, X[te])).max()
```

- `pred_contribs=True` — one value per feature for each row, plus a last column for the base.
- `C.sum(1)` — adds along each row: this must equal the prediction.

REAL GRAPH



Real graph from the script: speed, missing values, five models, and SHAP.

Quick Syntax Glossary — new or reinforced constructs

- `xgb.DMatrix(X, label=y)` — the library's data format.
- `xgb.train(params, d, rounds, evals=..., early_stopping_rounds=...)` — training.
- `model.predict(d, output_margin=True)` — raw log-odds.
- `getattr(obj, "name", None)` — an attribute, or None if it is not there.
- `np.nanmedian(a, axis=0)` — a median that ignores gaps.
- `tempfile.TemporaryDirectory()` — a folder removed at the end of the with block.
- `dict(BASE, tree_method="hist")` — a copy of a dictionary with extra settings.

End of Day 110 syntax reference. For the experiments and what they showed, see the Day 110 complete guide.